

```
"""
```

ربات حسابداری تلگرام برای تعمیرات و فروش موبایل

نیازمندی‌ها:

```
pip install python-telegram-bot anthropic python-dotenv
```

(.env فایل) متغیرهای محیطی مورد نیاز:

```
TELEGRAM_TOKEN=your_telegram_bot_token
```

```
ANTHROPIC_API_KEY=your_anthropic_api_key
```

```
ALLOWED_USER_ID=your_telegram_user_id (اختیاری - برای امنیت)
```

```
"""
```

```
import os
```

```
import json
```

```
import base64
```

```
import logging
```

```
from datetime import datetime
```

```
from pathlib import Path
```

```
from dotenv import load_dotenv
```

```
from telegram import Update, ReplyKeyboardMarkup, KeyboardButton
```

```
from telegram.ext import (
```

```
    Application, CommandHandler, MessageHandler,
```

```
    filters, ContextTypes, ConversationHandler
```

```
)
```

```
import anthropic
```

```
load_dotenv()
```

```
logging.basicConfig(
```

```
    format="%(asctime)s - %(name)s - %(levelname)s - %(message)s",
```

```
    level=logging.INFO
```

```
)
```

```
logger = logging.getLogger(__name__)
```

```
# ————— تنظیمات —————
```

```
TELEGRAM_TOKEN = os.getenv("TELEGRAM_TOKEN")
```

```
ANTHROPIC_KEY = os.getenv("ANTHROPIC_API_KEY")
```

```
ALLOWED_USER_ID = os.getenv("ALLOWED_USER_ID") # اختیاری
```

```
DATA_FILE = "accounting_data.json"
```

```
claude = anthropic.Anthropic(api_key=ANTHROPIC_KEY)
```

```
# ————— ذخیره / بارگذاری داده —————
```

```
def load_data() -> dict:
```

```

if Path(DATA_FILE).exists():
    with open(DATA_FILE, "r", encoding="utf-8") as f:
        return json.load(f)
return {
    "transactions": [], # درآمد و هزینه
    "customers": [], # مشتری‌های بدهکار
    "balances": [], # حساب با اشخاص دیگر
    "notes": [] # یادداشت‌های متفرقه
}

def save_data(data: dict):
    with open(DATA_FILE, "w", encoding="utf-8") as f:
        json.dump(data, f, ensure_ascii=False, indent=2)

# — Claude سیستم پرامپت برای —
SYSTEM_PROMPT = """
تو دستیار حسابداری هوشمند یک فروشگاه تعمیرات و فروش موبایل هستی.

وظایف اصلی تو:
1. ثبت درآمدها (فروش، تعمیر) و هزینه‌ها (قطعات، اجاره، حقوق و...).
2. ثبت مشتریانی که کارشان انجام شده ولی هنوز پول ندادند (بدهکاران).
3. ثبت حساب‌های جاری با افراد دیگر (شریک، تامین‌کننده، ...).
4. گزارش سود/زیان ماهانه.
5. پردازش تصویر (فاکتور، رسید، یادداشت دستنویس).

قوانین پاسخ:
- همیشه فارسی پاسخ بده.
- اگر اطلاعات ناقص است، سوال بپرس.
- همیشه پس از ثبت، خلاصه ثبت شده را تایید کن.
- اعداد را با فرمت واضح (مثلاً ۵۰۰,۰۰۰ تومان) نمایش بده.

داده‌های فعلی سیستم:
{data_summary}

شروع کن JSON وقتی کاربر اطلاعاتی می‌دهد که باید ثبت شود، ساخت را با یک بلوک
```json
{{
 "action": "record_transaction" | "record_customer_debt" | "record_balance" | "n
 "data": {{ ... }}
}}
```
سپس توضیح فارسی بده.

action برای هر JSON فرمت‌های:

record_transaction:
{{

```

```

    "type": "income" | "expense",
    "amount": عدد,
    "category": "سایر" | "حقوق" | "اجاره" | "قطعات" | "فروش" | "تعمیر",
    "description": "توضیح",
    "date": "YYYY-MM-DD"
}}

```

record_customer_debt:

```

{{
    "name": "نام مشتری",
    "phone": "شماره (اختیاری)",
    "amount": عدد,
    "service": "خدمت انجام شده",
    "date": "YYYY-MM-DD",
    "status": "unpaid" | "paid"
}}

```

record_balance:

```

{{
    "person": "نام شخص",
    "amount": عدد,
    "type": "owed_to_me" | "i_owe",
    "description": "توضیح",
    "date": "YYYY-MM-DD"
}}
"""

```

def build_data_summary(data: dict) -> str:

today = datetime.now()

month_str = today.strftime("%Y-%m")

income = sum(t["amount"] for t in data["transactions"]

if t["type"] == "income" and t["date"].startswith(month_str))

expense = sum(t["amount"] for t in data["transactions"]

if t["type"] == "expense" and t["date"].startswith(month_str))

unpaid_customers = [c for c in data["customers"] if c.get("status") == "unpai

total_unpaid = sum(c["amount"] for c in unpaid_customers)

owed_to_me = sum(b["amount"] for b in data["balances"] if b["type"] == "owed_

i_owe = sum(b["amount"] for b in data["balances"] if b["type"] == "i_owe

return f"""

ماه جاری ({month_str}):

- تومان درآمد: {income:,}

- تومان هزینه: {expense:,}

- تومان سود خالص: {income - expense:,}

بدهکاران (کار انجام شده - پول نگرفته):

- نفر: {len(unpaid_customers)}

- تومان: {total_unpaid:,}

- لیست: {'', '.join(f"{c['name']}{c['amount']:,})" for c in unpaid_customers[:5])}

حساب با دیگران:

- تومان: {owed_to_me:,} طلب از دیگران

- تومان: {i_owe:,} بدهی به دیگران

"""

————— و ذخیره داده Claude پردازش پاسخ

def process_claude_response(response_text: str, data: dict) -> tuple[str, dict]:

"""و ذخیره در داده‌ها Claude از پاسخ JSON استخراج"""

import re

json_match = re.search(r"```json\s*(.*?)\s*```", response_text, re.DOTALL)

clean_text = re.sub(r"```json\s*.*?\s*```", "", response_text, flags=re.DOTALL)

if not json_match:

return clean_text, data

try:

action_data = json.loads(json_match.group(1))

action = action_data.get("action", "no_action")

payload = action_data.get("data", {})

if action == "record_transaction":

payload.setdefault("date", datetime.now().strftime("%Y-%m-%d"))

data["transactions"].append(payload)

elif action == "record_customer_debt":

payload.setdefault("date", datetime.now().strftime("%Y-%m-%d"))

payload.setdefault("status", "unpaid")

آپدیت اگر مشتری وجود دارد

existing = next((c for c in data["customers"]

if c["name"] == payload["name"]), None)

if existing:

existing.update(payload)

else:

data["customers"].append(payload)

elif action == "record_balance":

payload.setdefault("date", datetime.now().strftime("%Y-%m-%d"))

existing = next((b for b in data["balances"]

if b["person"] == payload["person"]), None)

if existing:

existing.update(payload)

```

        else:
            data["balances"].append(payload)

        save_data(data)
    except (json.JSONDecodeError, KeyError) as e:
        logger.warning(f"JSON parse error: {e}")

    return clean_text, data

# ————— هاندلرهای تلگرام —————
async def start(update: Update, context: ContextTypes.DEFAULT_TYPE):
    keyboard = [
        [KeyboardButton("📊 گزارش ماهانه"), KeyboardButton("👥 بدهکاران")],
        [KeyboardButton("💰 ثبت درآمد"), KeyboardButton("💵 ثبت هزینه")],
        [KeyboardButton("⚖️ حساب با دیگران"), KeyboardButton("✅ پرداخت مشتری")],
    ]
    markup = ReplyKeyboardMarkup(keyboard, resize_keyboard=True)
    await update.message.reply_text(
        "👋 سلام!\n\nمن دستیار حسابداری فروشگاه موبایل شما هستم\n"
        "می‌تونید:\n"
        "• متن بنویسید (مثلاً: «صادق ۳۰۰ تومان بدهکاره»)\n"
        "• عکس فاکتور یا رسید بفرستید\n"
        "• از دکمه‌های پایین استفاده کنید\n\n"
        "چطور می‌تونم کمک کنم؟",
        reply_markup=markup
    )

async def handle_text(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if ALLOWED_USER_ID and str(update.effective_user.id) != ALLOWED_USER_ID:
        await update.message.reply_text("🚫 دسترسی مجاز نیست.")
        return

    user_text = update.message.text
    data = load_data()

    # دکمه‌های سریع
    if user_text == "📊 گزارش ماهانه":
        user_text = "گزارش کامل سود و زیان ماه جاری رو بده"
    elif user_text == "👥 بدهکاران":
        user_text = "لیست تمام مشتری‌هایی که پول ندادن رو نشون بده"
    elif user_text == "⚖️ حساب با دیگران":
        user_text = "وضعیت حساب من با همه افراد رو نشون بده"

    await update.message.chat.send_action("typing")

    try:
        response = claude.messages.create(

```

```

        model="claude-sonnet-4-20250514",
        max_tokens=1000,
        system=SYSTEM_PROMPT.format(data_summary=build_data_summary(data)),
        messages=[{"role": "user", "content": user_text}]
    )
    reply_text = response.content[0].text
    clean_reply, data = process_claude_response(reply_text, data)
    await update.message.reply_text(clean_reply)

except Exception as e:
    logger.error(f"Claude API error: {e}")
    await update.message.reply_text("خطایی رخ داد. لطفاً دوباره امتحان کنید ❌")

async def handle_photo(update: Update, context: ContextTypes.DEFAULT_TYPE):
    if ALLOWED_USER_ID and str(update.effective_user.id) != ALLOWED_USER_ID:
        await update.message.reply_text("دسترسی مجاز نیست 🚫")
        return

    await update.message.chat.send_action("typing")
    data = load_data()

    photo = update.message.photo[-1]
    caption = update.message.caption or "پیل کن و هر اطلاعات مالی که داره رو ثبت کن"

    try:
        photo_file = await photo.get_file()
        photo_bytes = await photo_file.download_as_bytearray()
        photo_b64 = base64.standard_b64encode(photo_bytes).decode("utf-8")

        response = claude.messages.create(
            model="claude-sonnet-4-20250514",
            max_tokens=1000,
            system=SYSTEM_PROMPT.format(data_summary=build_data_summary(data)),
            messages=[{
                "role": "user",
                "content": [
                    {
                        "type": "image",
                        "source": {
                            "type": "base64",
                            "media_type": "image/jpeg",
                            "data": photo_b64
                        }
                    },
                    {"type": "text", "text": caption}
                ]
            }]
        )

```

```

    )
    reply_text = response.content[0].text
    clean_reply, data = process_claude_response(reply_text, data)
    await update.message.reply_text(clean_reply)

except Exception as e:
    logger.error(f"Photo handling error: {e}")
    await update.message.reply_text("❌ دوباره امتحان کنید")

async def report_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """برای گزارش سریع دستور"""
    data = load_data()
    summary = build_data_summary(data)
    await update.message.reply_text(f"🇮🇷 **خلاصه حسابها**\n{summary}", parse_mode=HTML)

async def reset_command(update: Update, context: ContextTypes.DEFAULT_TYPE):
    """برای پاک کردن داده‌ها دستور"""
    if ALLOWED_USER_ID and str(update.effective_user.id) != ALLOWED_USER_ID:
        return
    save_data({"transactions": [], "customers": [], "balances": [], "notes": []})
    await update.message.reply_text("✅ تمام داده‌ها پاک شدند")

# — اجرای ربات —
def main():
    if not TELEGRAM_TOKEN or not ANTHROPIC_KEY:
        raise ValueError("TELEGRAM_TOKEN و ANTHROPIC_API_KEY باید در فایل .env باشند")

    app = Application.builder().token(TELEGRAM_TOKEN).build()

    app.add_handler(CommandHandler("start", start))
    app.add_handler(CommandHandler("report", report_command))
    app.add_handler(CommandHandler("reset", reset_command))
    app.add_handler(MessageHandler(filters.PHOTO, handle_photo))
    app.add_handler(MessageHandler(filters.TEXT & ~filters.COMMAND, handle_text))

    logger.info("ربات در حال اجراست...")
    app.run_polling(allowed_updates=Update.ALL_TYPES)

if __name__ == "__main__":
    main()

```